Лабораторная работа № 21: Исследование методов работы с потоками в Node.js.

Цель работы: Сформировать умение анализировать методы работы с потоками в Node.js.

Задание 1 (Базовый уровень)

Создание простого Readable-потока и чтение данных

Описание:
Необходимо создать простой Readable-поток, который генерирует данные по частям, и продемонстрировать чтение из него в flowing-режиме (событие `data`).

Как это должно работать:

1. Создаётся Readable-поток через `stream.Readable`
2. В методе `_read(size)` поток генерирует данные (например, строки с информацией о группе)
3. Подписываемся на события `data`, `end`, `error`
4. При каждом чанке выводим его в консоль
5. После завершения выводим сообщение об окончании

Содержимое чанков:

- Первый чанк: "Группа: ББМО-01-23"
- Второй чанк: "Студент: [Ваше ФИО]"
- Третий чанк: "Лабораторная работа: №21"
- Четвёртый чанк: "Тема: Потоки в Node.js"
- Далее — `null` (конец потока)

Пример работы:

```
text
```

```
$ node stream-readable.js

=== Демонстрация Readable-потока ===

[CHUNK 1] Группа: ББМО-01-23
[CHUNK 2] Студент: Иванов Иван
[CHUNK 3] Лабораторная работа: №21
[CHUNK 4] Тема: Потоки в Node.js

[END] Поток завершён
Всего получено чанков: 4
```

```
Общий размер данных: 98 байт
```

Что нужно сделать для отчёта:

- Сделать скриншот запуска скрипта
- Сделать скриншот вывода всех чанков
- Сделать скриншот события `end`
- Показать, что данные приходят по частям (чанки)

Задание 2 (Средний уровень)

Создание Writable-потока и запись данных

Описание:
Необходимо создать Writable-поток, который принимает данные и обрабатывает их (например, записывает в файл или выводит в консоль с префиксом).
Продемонстрировать работу метода `write()` и обработку backpressure.

Как это должно работать:

1. Создаётся Writable-поток через `stream.Writable`
2. В методе `_write(chunk, encoding, callback)` обрабатываем чанк:
    - Выводим в консоль с префиксом `[WRITE]`
    - Имитируем задержку (setTimeout)
    - Вызываем `callback()` после обработки
3. Записываем данные через `write()`
4. Проверяем возвращаемое значение `write()`:
    - `true` — буфер не переполнен
    - `false` — буфер переполнен, ждём `drain`
5. Подписываемся на события `drain`, `finish`, `error`

Демонстрация backpressure:

- Устанавливаем `highWaterMark: 16` (маленький буфер)
- Записываем много данных быстро
- Показываем, что `write()` возвращает `false`
- Ждём событие `drain` и продолжаем

Пример работы:

```
text
```

```
$ node stream-writable.js
```

=== Демонстрация Writable-потока ===

[WRITE] Группа: ББМО-01-23
write() вернул: true

[WRITE] Студент: Иванов Иван
write() вернул: true

[WRITE] Лабораторная работа: №21
write() вернул: false ← буфер переполнен!

[DRAIN] Буфер освобождён, продолжаем запись
[WRITE] Тема: Потоки в Node.js
write() вернул: true

[FINISH] Все данные записаны

Всего записано: 4 чанка

Что нужно сделать для отчёта:

- Скриншот запуска скрипта
- Скриншот записи данных
- Скриншот момента, когда `write()` вернул `false`
- Скриншот события `drain`
- Скриншот события `finish`

Задание 3 (Продвинутый уровень)

Использование pipe() и pipeline() для соединения потоков

Описание:
Изучить разницу между `pipe()` и `pipeline()`. Продемонстрировать безопасную обработку ошибок и автоматическую очистку ресурсов.

Как это должно работать:

1. Демонстрация `pipe()`:
    - Создаётся Readable-поток (чтение файла)
    - Создаётся Writable-поток (запись в файл)
    - Соединяем через `pipe()`
    - Показываем, что при ошибке в одном из потоков остальные остаются открытыми
2. Демонстрация `pipeline()`:
    - Те же потоки соединяем через `pipeline()`

    - При ошибке все потоки автоматически закрываются
    - Единый колбэк для обработки завершения или ошибки
3. Сценарий с ошибкой:
    - Читаем несуществующий файл
    - Показываем разницу в поведении `pipe()` и `pipeline()`
4. Сценарий с успехом:
    - Копируем файл `input.txt` в `output.txt`
    - Сравниваем содержимое
5. Цепочка потоков:
    - `source.pipe(transform1).pipe(transform2).pipe(destination)`
    - Демонстрация последовательной обработки

Пример работы:

```
text
```

```
$ node stream-pipe.js

=== Демонстрация pipe() ===
Чтение: input.txt
Запись: output-pipe.txt
✔ Копирование завершено
Размер: 1024 байт

=== Демонстрация pipe() с ошибкой ===
Чтение: missing.txt
✖ Ошибка: ENOENT: no such file or directory
⚠ Внимание: pipe() не закрыл целевой поток!
⚠ Возможна утечка памяти.

=== Демонстрация pipeline() ===
Чтение: input.txt
Запись: output-pipeline.txt
✔ Копирование завершено

=== Демонстрация pipeline() с ошибкой ===
Чтение: missing.txt
✖ Ошибка: ENOENT: no such file or directory
✔ Все потоки автоматически закрыты
✔ Утечек нет

=== Цепочка потоков ===
input.txt → uppercase → reverse → output-chain.txt
```

```
✔ Обработка завершена
```

Что нужно сделать для отчёта:

- Скриншот успешного `pipe()`
- Скриншот `pipe()` с ошибкой (показать, что потоки не закрыты)

- Скриншот успешного `pipeline()`
- Скриншот `pipeline()` с ошибкой (показать автозакрытие)
- Скриншот цепочки потоков
- Скриншот содержимого выходных файлов

Задание 4 (Сложный уровень)

Создание Transform-потока и Object Mode

Описание:
Разработать собственный Transform-поток для преобразования данных. Изучить объектный режим (`objectMode: true`) и работу с асинхронными итераторами.

Как это должно работать:

1. Transform-поток `UpperCaseTransform`:
    - Принимает строки
    - Преобразует в верхний регистр
    - Добавляет префикс `[ВВМО-01-23]`
2. Transform-поток `JsonParserTransform`:
    - Принимает строки с JSON-объектами (по одной на строку)
    - Парсит их в объекты
    - Работает в `objectMode: true`
3. Transform-поток `FilterTransform`:
    - Фильтрует объекты по заданному условию
    - Например, оставляет только студентов группы ББМО-01-23
4. Демонстрация асинхронных итераторов:
    - Чтение потока через `for await (const chunk of readable)`
    - Императивная обработка каждого чанка
    - Сравнение с `pipe()`
5. Демонстрация Object Mode:
    - Поток генерирует объекты (не строки/Buffer)
    - Transform-поток обрабатывает объекты
    - Writable-поток принимает объекты
6. Цепочка:
    - `readable → upperCase → filter → writable`
    - Показываем, как данные преобразуются на каждом этапе

Пример работы:

```
text
```

```
$ node stream-transform.js
```

=== Демонстрация Transform-потока ===
Вход: "группа ббмо-01-23"
Выход: "[ВВМО-01-23] ГРУППА ББМО-01-23"

Вход: "студент иванов"
Выход: "[ВВМО-01-23] СТУДЕНТ ИВАНОВ"

=== Демонстрация Object Mode ===
[OBJECT] { id: 1, name: "Иван", group: "ББМО-01-23" }
[OBJECT] { id: 2, name: "Мария", group: "ББМО-02-23" }
[OBJECT] { id: 3, name: "Петр", group: "ББМО-01-23" }

После фильтрации (только ББМО-01-23):
[OBJECT] { id: 1, name: "Иван", group: "ББМО-01-23" }
[OBJECT] { id: 3, name: "Петр", group: "ББМО-01-23" }

=== Демонстрация async iterator ===
[ASYNC] Получен чанк 1: "Данные 1"
[ASYNC] Получен чанк 2: "Данные 2"
[ASYNC] Получен чанк 3: "Данные 3"
[ASYNC] Поток завершён

=== Цепочка Transform ===
Читаем → UpperCase → Filter → Записываем

✔ Обработано 10 строк, отфильтровано 7

Что нужно сделать для отчёта:

- Скриншот работы `UpperCaseTransform`
- Скриншот работы `JsonParserTransform`
- Скриншот работы `FilterTransform`
- Скриншот Object Mode
- Скриншот асинхронных итераторов
- Скриншот цепочки Transform-потоков

Задание 5 (Экспертный уровень)

Полноценный конвейер обработки данных с backpressure и мониторингом

Описание:
Создать полнофункциональный конвейер обработки данных, включающий:

- Генерацию данных (Readable)
- Несколько Transform-потоков
- Запись в файл (Writable)

- Управление backpressure
- Мониторинг прогресса
- Обработку ошибок
- Отмену по таймауту

Как это должно работать:

1. Источник данных (Readable):
    - Генерирует 10000 объектов-студентов
    - Каждый объект: `{ id, name, group, course, grade }`
    - Группа студентов — ББМО-01-23 (номер группы используется для seed)
    - Генерация происходит порциями (по 100 объектов)
2. Transform-потоки:
    - `ValidateTransform` — проверяет корректность данных
    - `EnrichTransform` — добавляет вычисляемые поля (например, `averageGrade`)
    - `FilterTransform` — оставляет только студентов с оценкой > 3
    - `FormatTransform` — форматирует объект в CSV-строку
3. Приёмник (Writable):
    - Записывает CSV в файл `students.csv`
    - Показывает прогресс (сколько записей обработано)
    - Обрабатывает backpressure
4. Управление backpressure:
    - `highWaterMark` для каждого потока
    - Обработка события `drain`
    - Логирование моментов приостановки
5. Мониторинг:
    - Прогресс-бар в консоли
    - Счётчики: обработано, отфильтровано, ошибок
    - Время выполнения
    - Скорость обработки (записей/сек)
6. Обработка ошибок:
    - `pipeline()` для безопасного соединения
    - Слушатель `error` на каждом потоке
    - Логирование ошибок в `errors.log`
7. Отмена по таймауту:
    - Если обработка занимает больше 10 секунд — отмена
    - Использование `AbortController` или `destroy()`
    - Корректное освобождение ресурсов
8. Асинхронные итераторы:
    - Демонстрация альтернативного подхода через `for await`
    - Сравнение с `pipeline()`

Пример работы:

```
text
```

```
$ node stream-pipeline.js

=== Конвейер обработки данных (группа ББМО-01-23) ===
Источник: 10000 студентов
Фильтр: оценка > 3

[PROGRESS] ████████░░░░░░░░░░░░ 40% | 4000/10000 | 850 записей/сек
[PROGRESS] ████████████████░░░░ 80% | 8000/10000 | 920 записей/сек
[PROGRESS] ████████████████████ 100% | 10000/10000 | 950 записей/сек

=== Результаты ===
Всего сгенерировано:    10000
Прошло валидацию:       9987
Обогащено:              9987
Отфильтровано:          7512
Записано в CSV:         7512
Ошибок:                 13
Время выполнения:       10.5 сек
Средняя скорость:       950 записей/сек

Файл: students.csv (2.3 МБ)

=== Демонстрация backpressure ===
[BACKPRESSURE] Writable переполнен на 5000-й записи
[BACKPRESSURE] Пауза 50 мс
[DRAIN] Продолжаем запись

=== Отмена по таймауту ===
Запуск с таймаутом 1 сек...
✖ Таймаут! Обработка отменена
✔ Все потоки корректно закрыты
✔ Ресурсы освобождены

=== Сравнение с async iterator ===
[ASYNC] Обработано: 7512 записей за 10.2 сек

Скорость: 736 записей/сек
```

Содержимое `students.csv`:

```
text

id,name,group,course,grade,averageGrade
1,Иван Иванов,ББМО-01-23,2,4.5,4.3
2,Мария Петрова,ББМО-01-23,2,3.8,3.9

...
```

Содержимое `errors.log`:

```
text
```

```
[2024-01-15 14:30:25] [ББМО-01-23] Ошибка валидации: id=123, отсутствует name
[2024-01-15 14:30:26] [ББМО-01-23] Ошибка валидации: id=456, некорректный grade
...
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта
- Скриншот запуска конвейера
- Скриншот прогресс-бара
- Скриншот итоговых результатов
- Скриншот содержимого `students.csv`
- Скриншот демонстрации backpressure
- Скриншот отмены по таймауту
- Скриншот содержимого `errors.log`
- Скриншот сравнения с async iterator

Фиксация изменений

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request